

BÀI THỰC HÀNH PHẦN MULTI-CLASS SUPPORT VECTOR MACHINE

Machine Learning

Mục tiêu bài thực hành

Sau bài thực hành này, học viên cần:

- Hiểu cách mở rộng SVM từ bài toán nhị phân sang bài toán phân loại đa lớp.
- Hiểu và cài đặt được SVM đa lớp theo hai hướng:
 - Crammer–Singer (Dual + Quadratic Programming).
 - Dạng unconstrained với multiclass hinge loss.
- Hiểu ý nghĩa hình học của margin đa lớp và slack variable.
- So sánh giữa cách cài đặt thủ công và thư viện chuẩn (`scikit-learn`).

Chú ý: Trong các nội dung dưới đây, để đơn giản hóa công thức, chúng ta mặc định $w = (b, w)^T$ và $x = (1, x)^T$, tức là các công thức coi như đã có bias. Phần lập trình các bạn cần dựa vào các phần trước để tự bổ sung bias, nếu không kết quả sẽ bị lệch.

Kiến thức cần nắm trước

- Support Vector Machine nhị phân (hard margin, soft margin).
- Lagrangian, duality, điều kiện KKT (mức khái niệm).
- Gradient descent.
- Lập trình Python với NumPy, Matplotlib.

1 Ví dụ 1: SVM đa lớp theo Crammer–Singer (Dual + QP)

1.1 Mô tả dữ liệu

- Không gian đặc trưng: $\mathbb{R}^2$.
- Số lớp: $K = 3$.
- Mỗi lớp gồm 100 mẫu, sinh ngẫu nhiên theo phân phối Gaussian.

1.2 Sinh dữ liệu

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

n_per_class = 100
means = [[0,0], [3,3], [0,4]]
cov = [[0.4,0], [0,0.4]]
```

```

X_list, y_list = [], []

for k, mu in enumerate(means):
    Xk = np.random.multivariate_normal(mu, cov, n_per_class)
    yk = np.full(n_per_class, k)
    X_list.append(Xk)
    y_list.append(yk)

X = np.vstack(X_list)
y = np.concatenate(y_list)

n, d = X.shape
K = len(np.unique(y))

```

1.3 Trực quan hóa phân bố dữ liệu

```

plt.figure(figsize=(6,5))
for k in range(K):
    plt.scatter(X[y==k,0], X[y==k,1], label=f"Class {k}", alpha=0.7)
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.title("Synthetic 3-class dataset")
plt.grid(True)
plt.show()

```

1.4 Nhắc lại bài toán Crammer–Singer (liên hệ code)

Primal (soft-margin, không bias để đơn giản hoá triển khai). Với dữ liệu (x_i, y_i) , $y_i \in \{0, 1, \dots, K-1\}$, đặt $W = [w_0, \dots, w_{K-1}]$:

$$\begin{aligned}
 \min_{W, \xi} \quad & \frac{1}{2} \sum_{k=0}^{K-1} \|w_k\|^2 + C \sum_{i=1}^n \xi_i \\
 \text{s.t.} \quad & w_{y_i}^\top x_i \geq w_k^\top x_i + 1 - \xi_i, \quad \forall i, \forall k \neq y_i; \quad \xi_i \geq 0.
 \end{aligned}$$

Dual (dạng Quadratic Programming). Giới thiệu biến đối ngẫu $\alpha_{i,k} \geq 0$ cho các ràng buộc $w_{y_i}^\top x_i - w_k^\top x_i \geq 1 - \xi_i$ với $k \neq y_i$. Ta thu được bài toán dual chuẩn QP với các ràng buộc:

$$\sum_{k \neq y_i} \alpha_{i,k} \leq C, \quad \forall i; \quad \alpha_{i,k} \geq 0.$$

Để giải bằng `cvxopt.solvers.qp`, ta cần đưa dual về dạng **minimize** chuẩn:

$$\min_{\alpha} \quad \frac{1}{2} \alpha^\top P \alpha + q^\top \alpha \quad \text{s.t.} \quad G\alpha \leq h, \quad A\alpha = b$$

trong đó $\alpha \in \mathbb{R}^m$ là vector hoá từ các phần tử $\alpha_{i,k}$ (chỉ lấy các cặp $k \neq y_i$), $m = n(K-1)$.

Liên hệ với dual theo Lagrange. Dual thường được viết dưới dạng **maximize**:

$$\max_{\alpha} \quad \mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top P \alpha$$

với cùng các ràng buộc. Khi chuyển sang dạng **minimize** cho qp, ta đổi dấu hạng tuyến tính:

$$\min_{\alpha} \quad \frac{1}{2} \alpha^\top P \alpha - \mathbf{1}^\top \alpha$$

tức là trong code sẽ có $q = -\mathbf{1}$.

1.5 Chuẩn bị ánh xạ chỉ số (i,k) và vector hoá biến đối ngẫu

Trong Crammer–Singer, với mỗi mẫu i ta có biến $\alpha_{i,k}$ cho mọi lớp $k \neq y_i$. Để giải QP bằng thư viện số, ta cần **vector hoá** toàn bộ các $\alpha_{i,k}$ thành một vector duy nhất:

$$\alpha = [\alpha_{(i_1, k_1)}, \alpha_{(i_2, k_2)}, \dots, \alpha_{(i_m, k_m)}]^\top \in \mathbb{R}^m, \quad m = n(K-1).$$

Ta tạo một danh sách pairs chứa toàn bộ các cặp chỉ số hợp lệ:

$$\text{pairs} = \{(i, k) \mid i \in \{0, \dots, n-1\}, k \in \{0, \dots, K-1\}, k \neq y_i\}.$$

Khi đó:

- Mỗi phần tử $\text{pairs}[t] = (i, k)$ tương ứng với biến $\alpha_{i,k}$.
- Vector hoá: $\alpha[t] \equiv \alpha_{i,k}$.
- Nhờ ánh xạ này, ta xây dựng được các ma trận P, q, G, h, A, b theo đúng thứ tự biến.

```
pairs = [(i, k) for i in range(n) for k in range(K) if k != y[i]]
m = len(pairs)
```

1.6 Xây dựng bài toán QP cho cvxopt: ý nghĩa P, q, G, h, A, b

Dạng chuẩn của cvxopt.solvers.qp. Hàm $\text{qp}(P, q, G, h, A, b)$ giải bài toán:

$$\min_{\alpha} \frac{1}{2} \alpha^\top P \alpha + q^\top \alpha \quad \text{s.t.} \quad G \alpha \leq h, \quad A \alpha = b$$

hoặc nếu ta muốn ràng buộc dạng “ $\leq$ ” theo từng mẫu, có thể dùng thủ thuật đưa vào G, h hoặc dùng A, b theo cấu trúc phù hợp. Trong bài này, ta dùng:

- G, h để áp đặt $\alpha \geq 0$,
- một ma trận kiểu “hàng theo mẫu” để áp đặt $\sum_{k \neq y_i} \alpha_{i,k} \leq C$.

(1) Vector tuyến tính q . Từ dual dạng minimize:

$$\min \frac{1}{2} \alpha^\top P \alpha - \mathbf{1}^\top \alpha,$$

suy ra:

$$q = -\mathbf{1}.$$

Trong code: $q = -\text{np.ones}(m)$.

(2) Ràng buộc không âm $\alpha \geq 0$: ma trận G, h . Điều kiện $\alpha_{i,k} \geq 0$ với mọi biến tương đương:

$$-I \alpha \leq 0.$$

Do đó:

$$G = -I_m, \quad h = \mathbf{0}.$$

Trong code: $G = -\text{np.eye}(m), h = \text{np.zeros}(m)$.

(3) **Ràng buộc tổng theo mỗi mẫu:** $\sum_{k \neq y_i} \alpha_{i,k} \leq C$. Với mỗi mẫu i , cộng các biến thuộc mẫu đó:

$$\sum_{k \neq y_i} \alpha_{i,k} \leq C.$$

Ta tạo ma trận $A_{\leq} \in \mathbb{R}^{n \times m}$ sao cho:

$$(A_{\leq} \alpha)_i = \sum_{k \neq y_i} \alpha_{i,k}.$$

Cụ thể:

$$(A_{\leq})_{i,t} = 1 \text{ nếu } \text{pairs}[t] = (i, k) \text{ (với bất kỳ } k \neq y_i), \text{ ngược lại } 0.$$

và

$$A_{\leq} \alpha \leq C \mathbf{1}.$$

Trong code, ma trận này được tạo bằng: $A[i, \text{idx}] = 1$ nếu $\text{pairs}[\text{idx}]$ có cùng i , và $b = C * \text{np.ones}(n)$.

1.7 Công thức tổng quát của ma trận P trong Dual Crammer–Singer

Nhắc lại dual Crammer–Singer (dạng minimize). Sau khi chuyển về dạng **minimize** để giải bằng QP, bài toán dual có dạng:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \alpha^{\top} P \alpha - \mathbf{1}^{\top} \alpha \\ \text{s.t.} \quad & \alpha_{i,k} \geq 0, \quad \sum_{k \neq y_i} \alpha_{i,k} \leq C, \forall i. \end{aligned}$$

Cấu trúc tổng quát của P . Xét hai biến đối ngẫu bất kỳ $\alpha_{i,k}$ và $\alpha_{j,l}$ (với $k \neq y_i, l \neq y_j$). Phần tử tương ứng của ma trận P được xác định bởi:

$$P_{(i,k),(j,l)} = (\mathbb{I}[y_i = y_j] - \mathbb{I}[y_i = l] - \mathbb{I}[y_j = k] + \mathbb{I}[k = l]) \langle x_i, x_j \rangle$$

Trong đó:

- $\mathbb{I}[\cdot]$ là hàm chỉ thị (bằng 1 nếu mệnh đề đúng, ngược lại bằng 0).
- $\langle x_i, x_j \rangle$ là tích vô hướng (trường hợp tuyến tính).

Giải thích từng thành phần. Hệ số trước $\langle x_i, x_j \rangle$ xuất phát từ việc:

- mỗi $\alpha_{i,k}$ tác động **tăng** w_{y_i} ,
- đồng thời tác động **giảm** w_k ,

nên khi khai triển $\sum_k \|w_k\|^2$ sẽ xuất hiện các hạng:

$$\langle x_i, x_j \rangle (\delta_{y_i=y_j} - \delta_{y_i=l} - \delta_{y_j=k} + \delta_{k=l}).$$

Ý nghĩa hình học. Ma trận P mã hoá:

- tương tác **giữa các mẫu khác nhau** ($i \neq j$),
- tương tác **giữa các lớp sai khác nhau** ($k \neq l$),

tạo nên một **QP toàn cục** thay vì các bài toán nhị phân độc lập như OvR.

Cách lắp đặt P tổng quát trong code.

```
P = np.zeros((m, m))
for p, (i, k) in enumerate(pairs):
    for q, (j, l) in enumerate(pairs):
        val = 0.0
        if y[i] == y[j]:
            val += 1
        if y[i] == 1:
            val -= 1
        if y[j] == k:
            val -= 1
        if k == 1:
            val += 1
        P[p, q] = val * np.dot(X[i], X[j])
```

Ghi chú:

- Đoạn code này triển khai đúng công thức tổng quát của P cho SVM Crammer–Singer tuyến tính.
- Với kernel, chỉ cần thay $\langle x_i, x_j \rangle$ bằng $K(x_i, x_j)$.

```
from cvxopt import matrix, solvers
solvers.options['show_progress'] = False

C = 1.0

pairs = [(i,k) for i in range(n) for k in range(K) if k != y[i]]
m = len(pairs)

P = np.zeros((m,m))
for p,(i,k) in enumerate(pairs):
    for q,(j,l) in enumerate(pairs):
        if i == j:
            if k == 1:
                P[p,q] = np.dot(X[i], X[j])
            else:
                P[p,q] = -np.dot(X[i], X[j])

q = -np.ones(m)
G = -np.eye(m)
h = np.zeros(m)

A = np.zeros((n,m))
for idx,(i,k) in enumerate(pairs):
    A[i,idx] = 1
b = C * np.ones(n)

solution = solvers.qp(
    matrix(P), matrix(q),
    matrix(G), matrix(h),
    matrix(A), matrix(b)
)

alpha = np.array(solution['x']).flatten()
```

1.8 Support Vectors và liên hệ điều kiện KKT

Điều kiện KKT trong Crammer–Singer. Với mỗi ràng buộc:

$$w_{y_i}^\top x_i \geq w_k^\top x_i + 1 - \xi_i,$$

điều kiện bổ sung KKT cho biến đối ngẫu $\alpha_{i,k}$ là:

$$\alpha_{i,k} (w_{y_i}^\top x_i - w_k^\top x_i - 1 + \xi_i) = 0$$

Diễn giải.

- Nếu ràng buộc **không hoạt động** (margin thỏa nghiêm ngặt):

$$w_{y_i}^\top x_i > w_k^\top x_i + 1 \Rightarrow \alpha_{i,k} = 0.$$

- Nếu ràng buộc **đang hoạt động** (điểm nằm trên biên hoặc trong margin):

$$w_{y_i}^\top x_i \leq w_k^\top x_i + 1 \Rightarrow \alpha_{i,k} > 0.$$

Định nghĩa Support Vector trong đa lớp. Một điểm x_i được gọi là **support vector** nếu tồn tại ít nhất một lớp $k \neq y_i$ sao cho:

$$\alpha_{i,k} > 0.$$

Nói cách khác:

$$x_i \text{ là support vector} \iff \exists k \neq y_i : \alpha_{i,k} > 0.$$

Triển khai thực tế trong code. Do sai số số học, ta không kiểm tra $\alpha_{i,k} > 0$ tuyệt đối, mà dùng một ngưỡng nhỏ ε :

$$\alpha_{i,k} > \varepsilon, \quad \varepsilon \approx 10^{-5}.$$

```
sv_idx = np.where(alpha > 1e-5)[0]
support_points = np.unique([pairs[i][0] for i in sv_idx])
```

Ý nghĩa hình học.

- Support vectors là các điểm **quyết định hình dạng biên phân lớp**.
- Các điểm khác ($\alpha_{i,k} = 0$ với mọi k) không ảnh hưởng đến nghiệm W .
- Đây là bản chất “sparse” của SVM, vẫn được giữ nguyên trong Crammer–Singer.

1.9 Khôi phục các vector w_k từ nghiệm đối ngẫu

Sau khi giải QP, ta thu được α (tức các $\alpha_{i,k}$). Từ điều kiện stationarity của Lagrangian theo w_k , ta có biểu diễn nghiệm primal:

$$w_k = \sum_{i: y_i=k} \sum_{l \neq k} \alpha_{i,l} x_i - \sum_{i: y_i \neq k} \alpha_{i,k} x_i$$

Diễn giải:

- Nếu x_i thuộc lớp k (tức $y_i = k$), nó đóng góp vào w_k qua các $\alpha_{i,l}$ với mọi lớp sai $l \neq k$.
- Nếu x_i không thuộc lớp k , nó làm giảm w_k với hệ số $\alpha_{i,k}$.

Vì ta đã vector hoá các $\alpha_{i,k}$ theo danh sách `pairs`, việc khôi phục $W = [w_0, \dots, w_{K-1}]$ được thực hiện bằng cách duyệt từng phần tử `pairs[t]=(i,k)`:

$$w_{y_i} \leftarrow w_{y_i} + \alpha_{i,k} x_i, \quad w_k \leftarrow w_k - \alpha_{i,k} x_i.$$

```
W = np.zeros((K,d))
for idx,(i,k) in enumerate(pairs):
    W[y[i]] += alpha[idx] * X[i]
    W[k]      -= alpha[idx] * X[i]
```

1.10 Vẽ vùng quyết định và support vectors

```
xx, yy = np.meshgrid(
    np.linspace(-2,5,300),
    np.linspace(-2,6,300)
)
grid = np.c_[xx.ravel(), yy.ravel()]
scores = grid @ W.T
pred = np.argmax(scores, axis=1)

plt.figure(figsize=(6,5))
plt.contourf(xx, yy, pred.reshape(xx.shape), alpha=0.25)

for k in range(K):
    plt.scatter(X[y==k,0], X[y==k,1], label=f"Class {k}")

sv_idx = np.where(alpha > 1e-5)[0]
sv_points = np.unique([pairs[i][0] for i in sv_idx])
plt.scatter(X[sv_points,0], X[sv_points,1],
            s=120, facecolors='none', edgecolors='k',
            label='Support Vectors')

plt.legend()
plt.title("Crammer--Singer SVM (Dual)")
plt.show()
```

1.11 Câu hỏi kiểm tra

- Vì sao mỗi điểm dữ liệu chỉ có một slack ξ_i ?
- Vai trò của tham số C trong bài toán là gì?

1.12 Yêu cầu phát triển

- Vẽ vùng quyết định và các support vectors.
- Tính độ chính xác trên tập huấn luyện.

Ghi chú. Trong các ví dụ minh họa trước, để giảm độ phức tạp tính toán, ma trận P có thể được rút gọn (chỉ xét $i = j$). Tuy nhiên, công thức tổng quát ở trên phản ánh đầy đủ bản chất của SVM Crammer–Singer tuyến tính và là nền tảng để mở rộng sang kernel SVM.

2 Ví dụ 2: Multiclass SVM với Hinge Loss

Phần này chúng ta lấy lại đoạn mã tạo dữ liệu như trong ví dụ 1, chỉ chuyển sang sử dụng phương pháp Hinge Loss để phân loại và so sánh kết quả với phương pháp đã dùng trong Ví dụ 1.

2.1 Multiclass hinge loss

Ta có hàm Loss dạng Hinge:

$$\ell_i(W) = \max_{k \neq y_i} \left(1 + w_k^\top x_i - w_{y_i}^\top x_i \right)_+$$

2.2 Giải thích gradient

Nếu k^* là lớp sai có margin lớn nhất:

$$\nabla_{w_{y_i}} = -x_i, \quad \nabla_{w_{k^*}} = x_i$$

2.3 Cài đặt Gradient Descent đầy đủ

```
W = np.zeros((K,d))
lr = 0.01
lam = 0.1
epochs = 300

for epoch in range(epochs):
    grad = lam * W
    for i in range(n):
        scores = X[i] @ W.T
        margins = scores - scores[y[i]] + 1
        margins[y[i]] = 0
        k_star = np.argmax(margins)
        if margins[k_star] > 0:
            grad[y[i]] -= X[i]
            grad[k_star] += X[i]
    W -= lr * grad
```

2.4 Câu hỏi

- Vì sao chỉ lớp sai “nguy hiểm nhất” ảnh hưởng gradient?
- So sánh tốc độ huấn luyện với ví dụ 1.

3 Ví dụ 3: SVM đa lớp với scikit-learn

```
from sklearn.svm import SVC
clf = SVC(kernel='linear', C=1.0)
clf.fit(X, y)
```

3.1 Yêu cầu

- Vẽ vùng quyết định.
- So sánh kết quả với hai ví dụ trước.

4 Đánh giá mô hình phân loại đa lớp

4.1 Các độ đo đánh giá

Đối với bài toán phân loại đa lớp, các độ đo thường dùng gồm:

- **Accuracy:**

$$\text{Accuracy} = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}}$$

- **Confusion Matrix:** bảng $K \times K$ với:

- Hàng: nhãn thực
- Cột: nhãn dự đoán

- **Precision, Recall, F1-score** (tính cho từng lớp và trung bình).

Tính Accuracy và Confusion Matrix


```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# ỨD đoán
y_pred = np.argmax(X @ W.T, axis=1)

# Accuracy
acc = accuracy_score(y, y_pred)
print("Accuracy:", acc)

# Confusion Matrix
cm = confusion_matrix(y, y_pred)
print("Confusion Matrix:\n", cm)

# Precision / Recall / F1
print(classification_report(y, y_pred))

```

Trực quan hóa Confusion Matrix

```

import seaborn as sns

plt.figure(figsize=(4,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[0,1,2],
            yticklabels=[0,1,2])
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Confusion Matrix")
plt.show()

```

4.2 Câu hỏi kiểm tra kiến thức

1. Accuracy cao có luôn đồng nghĩa với mô hình tốt không? Khi nào không?
2. Quan sát confusion matrix, lớp nào dễ bị nhầm lẫn nhất? Vì sao?
3. Khi dữ liệu mất cân bằng, nên ưu tiên độ đo nào?

5 Ví dụ 4–5–6: Bài tập tự giải

6 Ví dụ 4: Iris Dataset

6.1 Giới thiệu dữ liệu

- Số mẫu: 150
- Số lớp: 3
- Số thuộc tính: 4
- Bài toán phân loại hoa Iris

6.2 Đọc dữ liệu

```

from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)

```

6.3 Yêu cầu

- Chuẩn hóa dữ liệu
- Huấn luyện SVM đa lớp
- Đánh giá bằng accuracy và confusion matrix

7 Ví dụ 5: Wine Dataset

7.1 Giới thiệu dữ liệu

- Số mẫu: 178
- Số lớp: 3
- Số thuộc tính: 13
- Dữ liệu phân loại rượu vang

7.2 Đọc dữ liệu

```
from sklearn.datasets import load_wine
X, y = load_wine(return_X_y=True)
```

7.3 Yêu cầu

- Thử các giá trị C khác nhau
- So sánh kết quả OvR và linear SVM

7.4 Bài tập từ giải A: Dữ liệu tự chọn

- Lựa chọn một tập dữ liệu đa lớp phù hợp.
- Giải thích lựa chọn phương pháp huấn luyện.
- Đánh giá kết quả và thời gian huấn luyện.

7.5 Bài tập từ giải B: Phát triển chương trình

1. Cài đặt phiên bản **mini-batch gradient descent** cho multiclass hinge loss.
2. Thử nghiệm các giá trị learning rate khác nhau và nhận xét hiện tượng hội tụ.
3. So sánh thời gian huấn luyện giữa:
 - Dual (QP)
 - Primal (hinge loss + SGD)

7.6 Bài tập từ giải C: Kernel và mở rộng

1. Thay inner product $\langle x_i, x_j \rangle$ trong dual bằng kernel RBF:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

và khảo sát ảnh hưởng của γ .

2. So sánh Crammer–Singer kernel với OvR kernel SVM trong `scikit-learn`.
3. Áp dụng SVM đa lớp cho một tập dữ liệu lớn hơn (Digits hoặc Fashion-MNIST) và phân tích kết quả.

Kết luận

- One-vs-Rest là cách mở rộng tự nhiên từ SVM nhị phân, nhưng chưa phản ánh đầy đủ cấu trúc hình học đa lớp.
- Crammer–Singer chuẩn hóa khái niệm margin đa lớp, chỉ phạt lớp sai nguy hiểm nhất.
- Dạng hinge loss cho phép tối ưu hiệu quả trên dữ liệu lớn bằng các phương pháp gradient.
- Trong thực tế, việc lựa chọn:
 - Dual hay Primal
 - QP hay SGD

phụ thuộc vào quy mô dữ liệu và yêu cầu ứng dụng.